

## **Framework for Enterprise Architecture**

# **Enterprise Physics 101**



**John A. Zachman**  
**Zachman International**  
**2222 Foothill Blvd. Suite 337**  
**La Canada, Ca. 91011**  
**818-244-3763**

© 1990-2006 John A. Zachman, Zachman International

## **Preface**

This seminar is NOT about increasing the stock price by the close of market, Friday afternoon.

It IS about the laws of nature that determine the success of an Enterprise ... particularly, continuing success in the turbulent times of the Information Age.

It is a presentation on Physics ...

Enterprise Physics.

© 1990-2006 John A. Zachman, Zachman International

## Introduction

Enterprise Architecture presently appears to be a grossly misunderstood concept among management.

It is NOT an Information Technology issue.  
**It is an ENTERPRISE issue.**

It is likely perceived to be an Information Technology issue as opposed to a Management issue for two likely reasons:

- A. Awareness of it tends to surface in the Enterprise through the Information Systems community.
- B. Information Technology people seem to have the skills to do Enterprise Architecture if any Enterprise Architecture is being or is to be done.

## Origins of Ent. Arch.

Frederick Taylor "Principles of Scientific Management" 1911

Walter A. Shewhart "The Economic Control of Quality of Manufactured Product" 1931 (Dr. Edward Demming's Mgr.)

Peter Drucker "The Practice of Management" 1954

Jay Forrester "Industrial Dynamics" 1961

Peter Senge "The Fifth Discipline" 1990

Eric Helfert "Techniques of Financial Analysis" 1962

Robert Anthony "Planning and Control Systems: A Framework for Analysis" 1965

Sherman Blumenthal "Management Information Systems: A Framework for Planning and Development" 1969

Alvin Toffler "Future Shock" 1970

George Steiner "Comprehensive Managerial Planning" 1972  
Etc., etc., etc.

## "Enterprise"

There are two implications to the word "Enterprise":

### I. Scope

The broadest possible boundary of the Enterprise.  
The Enterprise in its entirety.  
Enterprise-wide in scope.  
The whole thing.

### II. Content

ENTERPRISE Architecture is for ENTERPRISES.  
Enterprise Architecture has nothing to do with the  
Enterprise's systems or its information technology  
(except as they may constitute Row 4 constraints).  
The end object is to engineer and manufacture  
the ENTERPRISE, NOT simply to build and run  
systems.

"ENTERPRISE" ACTUALLY MEANS "ENTERPRISE"

## The Information Age

"The next information revolution is well underway. But it is not happening where information scientists, information executives, and the information industry in general are looking for it. It is not a revolution in technology, machinery, techniques, software, or speed. It is a revolution in CONCEPTS."

*Peter Drucker. Forbes ASAP, August 24, 1998*

"Future Shock" (1970) - The rate of change.

"The Third Wave" (1980) - The structure of change.

"Powershift" (1990) - The culture of change.

*Alvin Toffler*

"We are living in an extraordinary moment in history. Historians will look back on our times, the 40-year time span between 1980 and 2020, and classify it among the handful of historic moments when humans reorganized their entire civilization around a new tool, a new idea."

*Peter Leyden. Minneapolis Star Tribune. June 4, 1995*

*"On the Edge of the Digital Age: The Historic Moment"*

# The Challenge

What is your strategy for addressing:

Orders of magnitude increases in complexity,  
and

Orders of magnitude increases in the rate of change?

Seven thousand years of history would suggest the only  
known strategy for addressing complexity and change is

ARCHITECTURE.

If it gets so complex you can't remember how it works,  
you have to write it down ... Architecture.

If you want to change how it works, you start with what  
you have written down ... Architecture.

The key to complexity and change: Architecture.

The question is: What is "Architecture,"  
Enterprise Architecture?

© 1990-2006 John A. Zachman, Zachman International

# Agenda

## **I. Global Environment**

- A. Escalating Complexity
- B. Escalating Rate of Change

## **II. Introduction to Enterprise Architecture**

- A. The Framework for Enterprise Architecture
- B. Enterprise Knowledgebase

## **III. Architecture Work**

- A. Three Variables
- B. End State Vision
- C. Enterprise Frustrations

## **IV. Primitives Versus Composites**

- A. Engineering versus Manufacturing
- B. Enterprise in Three Dimensions

## **V. Enterprise Engineering Design Objectives**

- A. Alignment, Integration, Flexibility, etc.
- B. Reducing Time-to-Market
- C. "Federated Architecture"

## **IV Frequently Asked Questions**

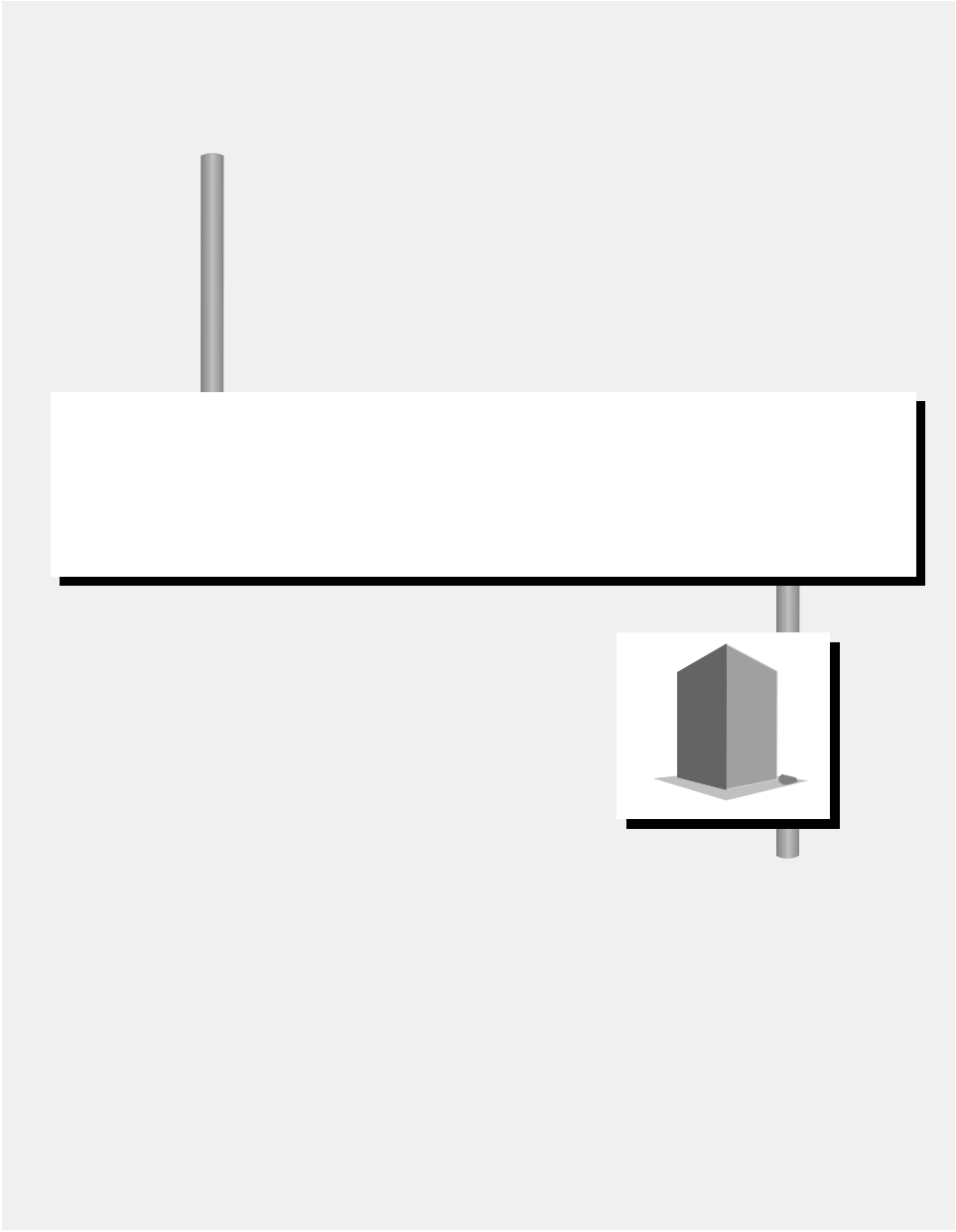
- A. Legacy Salvage
- B. Meta Frameworks

## **C. Value Proposition**

## **VII. Conclusions**

- A. A New Destination

© 2006 John A. Zachman, Zachman International



## Different Abstractions

| WHAT              | HOW               | WHERE          |
|-------------------|-------------------|----------------|
| Material          | Function          | Location       |
| Bill of Materials | Functional Specs  | Drawings       |
| Data Models       | Functional Models | Network Models |

© 1990-2006 John A. Zachman, Zachman International

## A Framework

|          | WHAT | HOW | WHERE |
|----------|------|-----|-------|
| OWNER    |      |     |       |
| DESIGNER |      |     |       |
| BUILDER  |      |     |       |
|          |      |     |       |

© 1990-2006 John A. Zachman, Zachman International

# A Framework

|                   | WHAT | HOW | WHERE |
|-------------------|------|-----|-------|
| SCOPE             |      |     |       |
| OWNER             |      |     |       |
| DESIGNER          |      |     |       |
| BUILDER           |      |     |       |
| OUT OF<br>CONTEXT |      |     |       |
| PRODUCT           |      |     |       |

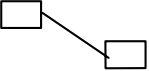
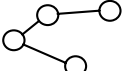
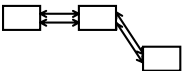
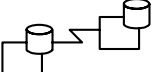
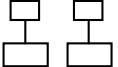
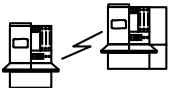
© 1990-2006 John A. Zachman, Zachman International

# A Framework

|                   | DATA | FUNCTION | NETWORK |
|-------------------|------|----------|---------|
| SCOPE             |      |          |         |
| BUSINESS<br>MODEL |      |          |         |
| SYSTEM<br>MODEL   |      |          |         |
| TECH<br>MODEL     |      |          |         |
| DETAIL<br>RPSNTNS |      |          |         |
| SYSTEM            |      |          |         |

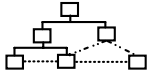
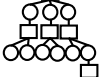
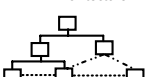
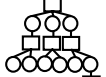
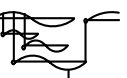
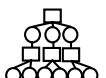
© 1990-2006 John A. Zachman, Zachman International

## ENTERPRISE ARCHITECTURE - A FRAMEWORK

|                                              | DATA <i>What</i>                                                                                                              | FUNCTION <i>How</i>                                                                                                          | NETWORK <i>Where</i>                                                                                                      |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| SCOPE<br>(CONTEXTUAL)                        | List of Things Important to the business<br> | List of Processes the Business Performs<br> | List of Locations in which the Business Operates                                                                          |
| <i>Planner</i>                               | ENTITY = Class of Business Thing                                                                                              | Process = Class of Business Process                                                                                          | Node = Major Business Location                                                                                            |
| BUSINESS MODEL<br>(CONCEPTUAL)               | e.g. Semantic Model<br>                      | e.g. Business Process Model<br>             | e.g. Business Logistics System<br>       |
| <i>Owner</i>                                 | Ent = Business Entity<br>ReIn = Business Relationship                                                                         | Proc = Bus Process<br>I/O = Bus Resources                                                                                    | Node = Business Location<br>Link = Business Linkage                                                                       |
| SYSTEM MODEL<br>(LOGICAL)                    | e.g. Logical Data Model<br>                  | e.g. Application Architecture<br>           | e.g. Distributed System Architecture<br> |
| <i>Designer</i>                              | Ent = Data Entity<br>ReIn = Data Relationship                                                                                 | Proc = Application Function<br>I/O = User Views                                                                              | Node = I/S Function (Processor, Storage, etc)<br>Link = Line Characteristics                                              |
| TECHNOLOGY MODEL<br>(PHYSICAL)               | e.g. Physical Data Model<br>                | e.g. System Design<br>                     | e.g. Technology Architecture<br>        |
| <i>Builder</i>                               | Ent = Segment/Table/etc.<br>ReIn = Pointer/Key/etc.                                                                           | Proc = Computer Function<br>I/O = Data Elements/Sets                                                                         | Node = Hardware/Systems Software<br>Link = Line Specifications                                                            |
| DETAILED REPRESENTATIONS<br>(OUT-OF-CONTEXT) | e.g. Data Definition<br>                   | e.g. Program<br>                          | e.g. Network Architecture<br>          |
| <i>Sub-Contractor</i>                        | Ent = Field<br>ReIn = Address                                                                                                 | Proc = Language Statement<br>I/O = Control Block                                                                             | Node = Address<br>Link = Protocol                                                                                         |
| FUNCTIONING ENTERPRISE                       | e.g. DATA                                                                                                                     | e.g. FUNCTION                                                                                                                | e.g. NETWORK                                                                                                              |



## ENTERPRISE ARCHITECTURE THE "OTHER THREE COLUMNS"

| PEOPLE                                                                                                                                                                   | TIME                                                                                                                                                                        | MOTIVATION                                                                                                                                                               |                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| List of Organizations Important to the Business<br><br>People = Major Organization Unit | List of Events/Cycles Significant to the Business<br><br>Time = Major Business Event/Cycle | List of Business Goals/Strategies<br><br>Ends/Mean = Major Business Goal/Strategy       | SCOPE (CONTEXTUAL)<br><br><i>Planner</i>                               |
| e.g., Work Flow Model<br><br>People = Organization Unit<br>Work = Work Product          | e.g., Master Schedule<br><br>Time = Business Event Cycle = Business Cycle                  | e.g., Business Plan<br><br>End = Business Objective<br>Means = Business Strategy        | BUSINESS MODEL (CONCEPTUAL)<br><br><i>Owner</i>                        |
| e.g., Human Interface Architecture<br><br>People = Role<br>Work = Deliverable           | e.g., Processing Structure<br><br>Time = System Event Cycle = Processing Cycle             | e.g., Business Rule Model<br><br>End = Structural Assertion<br>Means = Action Assertion | SYSTEM MODEL (LOGICAL)<br><br><i>Designer</i>                          |
| e.g., Presentation Architecture<br><br>People = User<br>Work = Screen Formats          | e.g., Control Structure<br><br>Time = Execute Cycle = Component Cycle                     | e.g., Rule Design<br><br>End = Condition<br>Means = Action                             | TECHNOLOGY MODEL (PHYSICAL)<br><br><i>Builder</i>                      |
| e.g., Security Architecture<br><br>People = Identity<br>Work = Job                    | e.g., Timing Definition<br><br>Time = Interrupt Cycle = Machine Cycle                    | e.g., Rule Specification<br><br>End = Sub-condition<br>Means = Step                   | DETAILED REPRESENTATIONS (OUT-OF-CONTEXT)<br><br><i>Sub-Contractor</i> |
| e.g., ORGANIZATION                                                                                                                                                       | e.g., SCHEDULE                                                                                                                                                              | e.g., STRATEGY                                                                                                                                                           | FUNCTIONING ENTERPRISE                                                 |

© 1990-2006 John A. Zachman, Zachman International

## Complexity and Change

### Complexity

Simplify ... Classification

Universal communication classification:

What, How, Where, Who, When, Why  
(Columns of the Framework)

### Rate of Change

A. Separate the Candidate Boundaries  
from the Business Concepts  
from the System Logic  
from the Technology Constructs  
from the Component Configurations  
from the Operations Reality  
... kind of like "layers".  
(Rows of the Framework)

B. Explicit models - baseline for managing change  
C. Mass Customization (See Engineering Design Objectives)

© 2006 John A. Zachman, Zachman International

# Architecture Is Architecture

I learned about architecture for Enterprises by looking at architecture for:

Airplanes, Buildings, Locomotives, Computers,  
Automobiles ... Complex Industrial Products

It is all the same ...

Bills of Material, Functional Specs, Drawings, etc.  
Requirements, Schematics, Blueprints, etc.

The Engineering Design Artifacts (the descriptive representations of anything) fall into a two dimensional classification system:

- A. The focus of the description  
(What, How, Where, Who, When, Why)
- B. The usage of the description  
(Owner, Designer, Builder)

I simply put Enterprise names on the same descriptive representations relevant for describing anything.

Why would anyone think that the descriptions of an Enterprise are going to be any different from the descriptions of everything else?

# The "System" IS the Enterprise

|                | What | How | Where | Who | When | Why |                |
|----------------|------|-----|-------|-----|------|-----|----------------|
| Planner        |      |     |       |     |      |     | Planner        |
| Owner          |      |     |       |     |      |     | Owner          |
| Designer       |      |     |       |     |      |     | Designer       |
| Builder        |      |     |       |     |      |     | Builder        |
| Sub-Contractor |      |     |       |     |      |     | Sub-Contractor |
| THE ENTERPRISE |      |     |       |     |      |     |                |

## The Framework Is a Schema

The Fmwrk is a two-dimensional classification system for ENTERPRISE descriptive representations NOT I/S.

The classification scheme for each axis grew up quite independently from the Framework application.

The classification for each axis is:

- a. Comprehensive
- b. Non-redundant

Therefore, each cell of the Framework is:

- a. Unique
  - b. Primitive (one single Abstraction by one single Perspective)
- and the total set of cells is complete.

The Framework logic is universal, independent of its application - totally neutral relative to methods/tools.

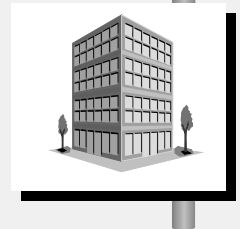
**The Framework is a "normalized" schema ...  
... NOT a matrix.**

That's what makes it a good analytical tool.

© 2001-2006 John A. Zachman, Zachman International

**Enterprise Architecture**

**Architecture Work**



© 2006 John A. Zachman, Zachman International

## Three Possibilities for Compromise

|            | What | How | Where | Who | When | Why |                   |
|------------|------|-----|-------|-----|------|-----|-------------------|
| Scope      |      |     |       |     |      |     | Visionaries       |
| Business   |      |     |       |     |      |     | Executive Leaders |
| System     |      |     |       |     |      |     | Architects        |
| Technology |      |     |       |     |      |     | Engineers         |
| Components |      |     |       |     |      |     | Implementers      |
| Risks      |      |     |       |     |      |     |                   |

2006 John A. Zachman, Zachman International

## Architecture Compromises

The Architecture Work alternatives are profoundly significant, because if, in your Enterprise Architecture (application development) methodology, you are not going to take the time and spend the money to build all the models and populate all of the possible intersections that constitute the total knowledgebase of the Enterprise, you have to understand the physics implications, that is, the risks of NOT building all the models and not populating all of the intersections.

Three possibilities for compromise can be seen in the Framework graphic itself.

# Less Than Enterprise-Wide Scope

|                | What   | How     | Where    | Who    | When | Why        |                |
|----------------|--------|---------|----------|--------|------|------------|----------------|
| Planner        |        |         |          |        |      |            | Planner        |
| Owner          |        |         |          |        |      |            | Owner          |
| Designer       |        |         |          |        |      |            | Designer       |
| Builder        |        |         |          |        |      |            | Builder        |
| Sub-Contractor |        |         |          |        |      |            | Sub-Contractor |
|                | Things | Process | Location | People | Time | Motivation |                |

Re: Any Cell

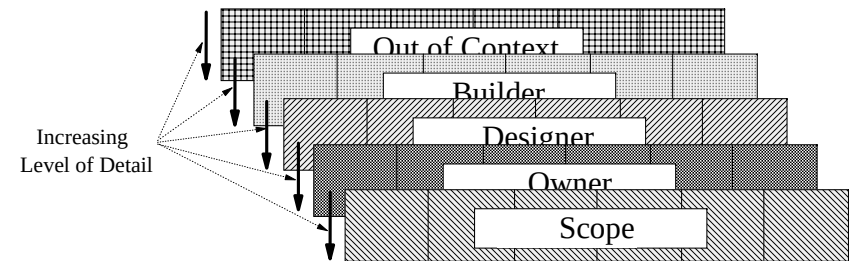
# Explicit .... or, Implicit

|                | What     | How     | Where    | Who    | When | Why        |                |
|----------------|----------|---------|----------|--------|------|------------|----------------|
| Planner        | Explicit |         |          |        |      |            | Planner        |
| Owner          |          |         |          |        |      |            | Owner          |
| Designer       |          |         |          |        |      |            | Designer       |
| Builder        |          |         |          |        |      |            | Builder        |
| Sub-Contractor |          |         |          |        |      |            | Sub-Contractor |
|                | Things   | Process | Location | People | Time | Motivation |                |

You are allowing anybody and everybody to make whatever assumptions they want to make about every Cell that has not been made explicit.  
**Erroneous Assumptions = Defects**

## Excruciating Level of Detail

Level of detail is a function of a Cell, NOT a Column.



## Less Than Excruciating Level of Detail

|                | What | How | Where | Who | When | Why |                |
|----------------|------|-----|-------|-----|------|-----|----------------|
| Planner        |      |     |       |     |      |     | Planner        |
| Owner          |      |     |       |     |      |     | Owner          |
| Designer       |      |     |       |     |      |     | Designer       |
| Builder        |      |     |       |     |      |     | Builder        |
| Sub-Contractor |      |     |       |     |      |     | Sub-Contractor |
| Things         |      |     |       |     |      |     |                |
| Process        |      |     |       |     |      |     |                |
| Location       |      |     |       |     |      |     |                |
| People         |      |     |       |     |      |     |                |
| Time           |      |     |       |     |      |     |                |
| Motivation     |      |     |       |     |      |     |                |

## Excruciating Level of Detail

Excruciating level of detail is not merely a technical, programmer's responsibility. At Row 5, because of the nature of the work, the model has to be intelligible to a machine and therefore, by definition will have to be at excruciating level of detail. However, EVERY Cell has a high level of detail, medium level of detail, excruciating level of detail. If the Row 1 and 2 models, for example, are to be used for something beyond planning, scoping, bounding, segmenting, etc., they will have to be defined at excruciating levels of detail. Otherwise, the Row 3, 4 and/or worst of all possible cases, Row 5 people will, by definition, have to make assumptions about what business the Enterprise is in and how it conceptually operates, and those assumptions then become the reality of the Functioning Enterprise.

## Basic "Physics"

1. If the Enterprise exists, ALL of the descriptive representations (models) exist ... by definition.

If they are not explicit, they are implicit (that is, you are making assumptions about them.)

2. High level descriptions (models) are good for planning, scoping, bounding, segmenting.

(High level descriptions are NO good for implementation.)

3. Narrow-in-scope descriptions are quick.

(Narrow in scope descriptions result in "stove pipes.")

## Implications of Compromise

1. The governance system should define, for the next planning period, which Cells or slivers of Cells you intend to make explicit. Any Cell (or portion of Cell) you do not make explicit is where there is risk of defects or discontinuity, entropy (disorder, energy not available for work).
2. High level descriptions (models) are good for planning, scoping, bounding, segmenting ... but not for implementing. If you do not define the excruciating level of detail, do you think it goes away?!! No. You are just making assumptions about it ... i.e. potential defects.
3. You can compromise Enterprise-wide integrity of some Columns of models in the interest of reducing the time it takes for implementation with impunity ... it is only inefficient, not optimal. However, compromising Enterprise-wide integrity in other Columns may directly, negatively impact management's performance.

© 2000-2006 John A. Zachman, Zachman International

## Two More Compromises

4. You can constrain or omit horizontal intersections in the metamodel to reduce the amount of work populating the Enterprise models at the expense of flexibility plus any horizontal intersection you do not populate is simply one implementation composite alternative you will not be able to support.
5. You can constrain or omit vertical intersections in the metamodel to reduce the amount of work at the expense of flexibility and traceability, that is, at the expense of "alignment" of the implementations with the intent of the Enterprise.

© 2000-2006 John A. Zachman, Zachman International



## Observations

After something is implemented, you cannot change its structural characteristics and you cannot create something out of nothing.

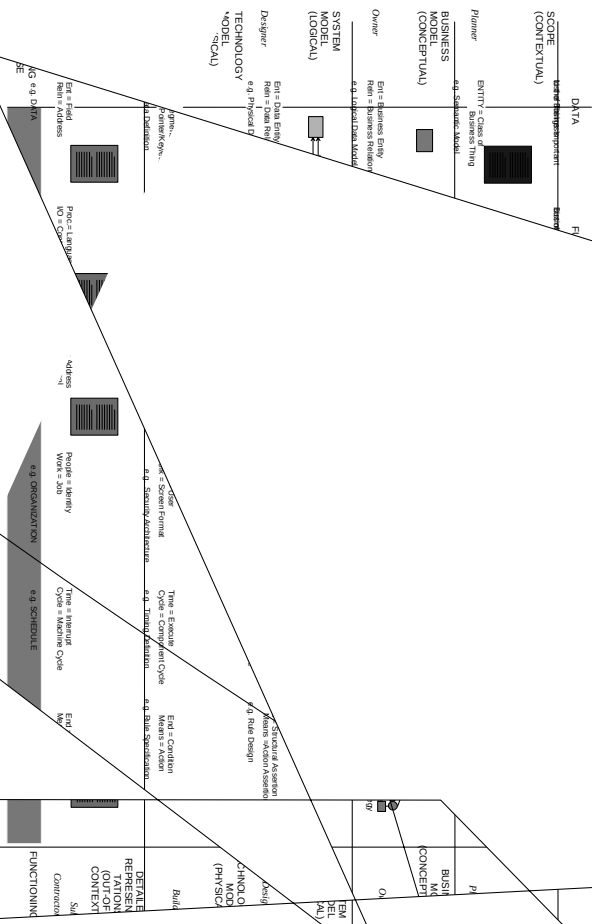
Do not lose sight of the fact that the end object is to produce a coherent, integrated ENTERPRISE, not simply to build and run systems. If you are simply building and running systems you are, by definition, DIS-integrating, SUB-optimizing, DIS-ordering, DE-normalizing the Enterprise.

## End State Vision

|                | What   | How     | Where    | Who    | When | Why        |                |
|----------------|--------|---------|----------|--------|------|------------|----------------|
| Planner        |        |         |          |        |      |            | Planner        |
| Owner          |        |         |          |        |      |            | Owner          |
| Designer       |        |         |          |        |      |            | Designer       |
| Builder        |        |         |          |        |      |            | Builder        |
| Sub-Contractor |        |         |          |        |      |            | Sub-Contractor |
|                | Things | Process | Location | People | Time | Motivation |                |

Some day  
You are going to wish you had  
all these models made explicit,  
Enterprise-wide,  
horizontally and vertically integrated,  
at excruciating level of detail !!!

# AN IMPLEMENTATION STRAT



Primitive Models  
Primitive Models

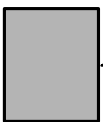
The long term Enterprise value  
lies in Enterprise "Engineering,"  
i.e. in the MODELS THEMSELVES!  
The "Knowledgebase" of the Enterprise  
(This is an Information Age idea!)

# Primitive Models

A "Primitive" Model is one that is comprised of elements from a single Framework Cell ... one single "abstraction" from one single "perspective."

|                | What   | How     | Where    | Who    | When | Why        |                |
|----------------|--------|---------|----------|--------|------|------------|----------------|
| Planner        |        |         |          |        |      |            | Owner          |
| Designer       |        |         |          |        |      |            | Designer       |
| Builder        |        |         |          |        |      |            | Builder        |
| Sub-Contractor |        |         |          |        |      |            | Sub-Contractor |
|                | Things | Process | Location | People | Time | Motivation |                |

**Primitive Models**

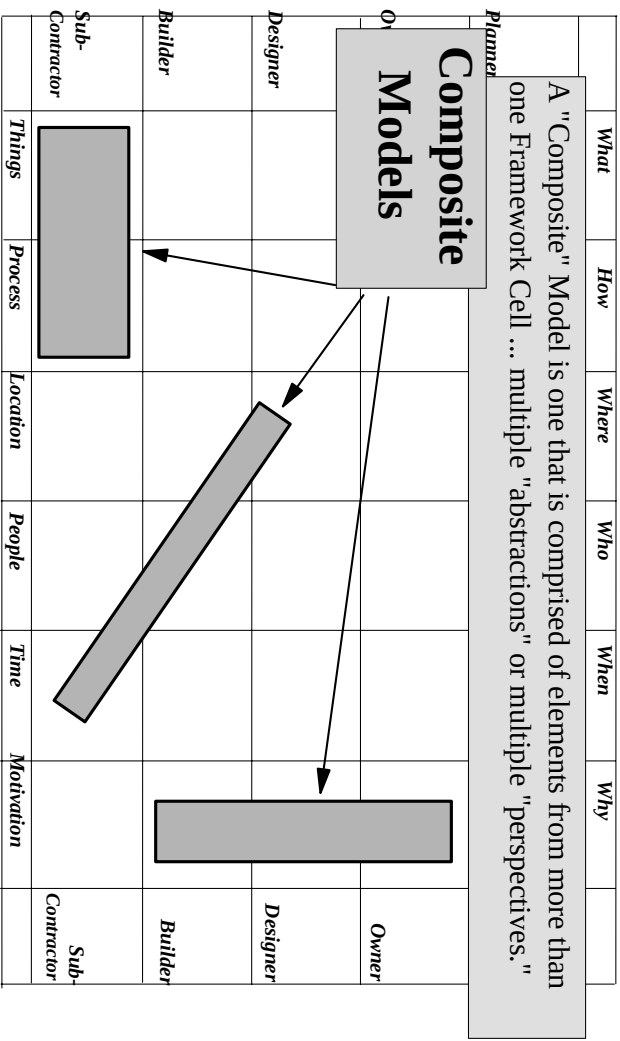


Enterprise Architecture

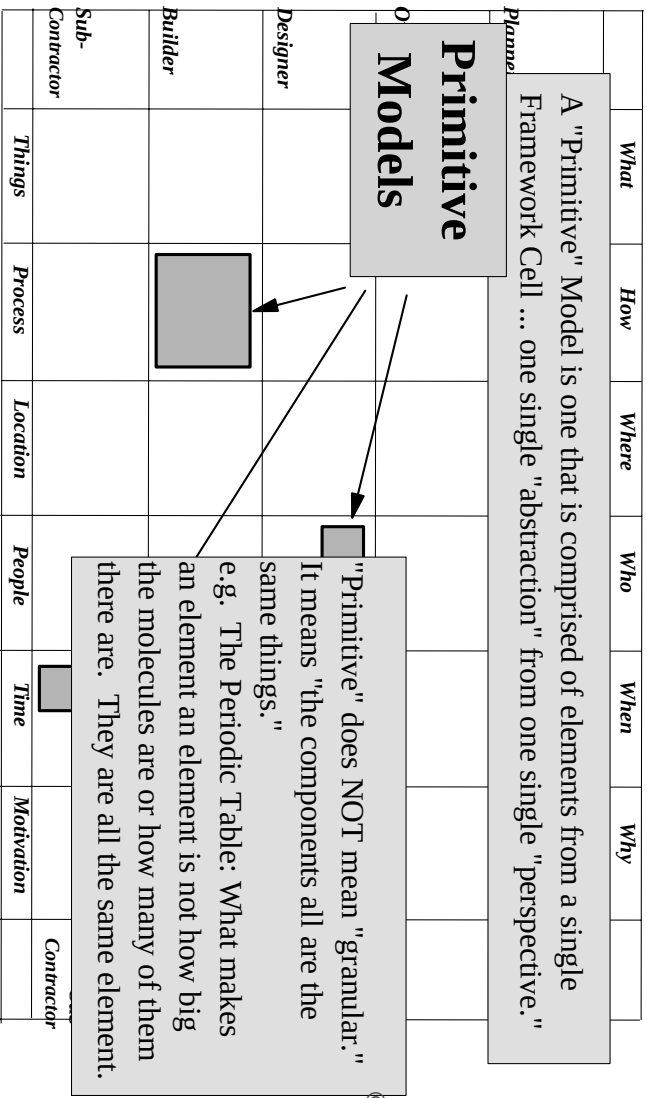
## Primitives Versus Composites



# Composite Models



# Primitive Models



# Architecture vs Implementation

If you are not building "primitive models," you are not doing Architecture, you are doing implementation.

Composite models are implementations.

Primitive models are Architecture.

Composite models should be created from primitive models.

If composite models are being created and no primitive models exist, then the composite model is likely being defined relative to a specific implementation (one component of one facet), not relative to *the Enterprise*. You are optimizing the *implementation* and SUB-OPTIMIZING the ENTERPRISE. It is a point-in-time solution. It is good only as long as nothing changes. The likelihood of it being reusable is low to zero. It is more "legacy."

## The "Silver Bullet"

Building implementations (composite models) and SAYING you are doing Enterprise Architecture (primitive models) is the worst possible architecture strategy.

© 2000-2006 John A. Zachman, Zachman International

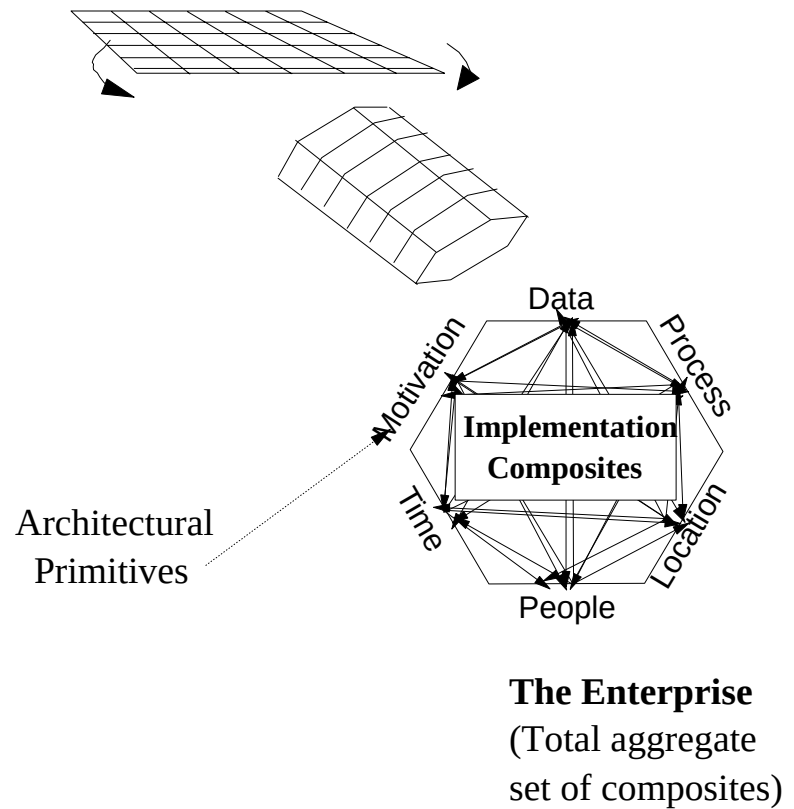
# Primitives vs Composites

|                | What                                                                                                                                                                                                                               | How     | Where    | Who    | When | Why        |                |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------|--------|------|------------|----------------|
| Planner        | You need Primitive Models for Architecture                                                                                                                                                                                         |         |          |        |      |            | Planner        |
| Owner          |                                                                                                                                                                                                                                    |         |          |        |      |            | Owner          |
| Designer       |                                                                                                                                                                                                                                    |         |          |        |      |            | Designer       |
| Builder        | You need Composite Models for Implementation<br>(For <i>architectured</i> implementations, composite models must be created from primitive models and diagonal composites from horizontally and vertically integrated primitives.) |         |          |        |      |            | Builder        |
| Sub-Contractor | Things                                                                                                                                                                                                                             | Process | Location | People | Time | Motivation | Sub-Contractor |

© 2006 John A. Zachman, Zachman International

# Architecture vs Implementation

From a fixed set of 36 Architectural Primitives, you can create a virtually infinite set of Implementation Composites.



## Primitives vs Composites

|                |                                                                                                                                                                                           |         |          |        |      |            |                |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------|--------|------|------------|----------------|
|                | Building Primitive Models:<br>The objective is ENGINEERING (Enterprise Architecture)                                                                                                      |         |          |        |      |            |                |
| Planner        | Building Composite Models:<br>The objective is MANUFACTURING (Implementation)                                                                                                             |         |          |        |      |            | Planner        |
| Owner          | Short Term Strategy:<br>Start Manufacturing. Worry about engineering later (legacy)                                                                                                       |         |          |        |      |            | Owner          |
| Designer       | Long Term Strategy:<br>Start Engineering. Minimize scrap and rework (architecture)                                                                                                        |         |          |        |      |            | Designer       |
| Builder        | Note: if you fabricate the Primitives and keep them in inventory, you can change the IS/IT strategy to "assemble to order" that is, assemble the Enterprise to order (mass customization) |         |          |        |      |            | Builder        |
| Sub-Contractor | Things                                                                                                                                                                                    | Process | Location | People | Time | Motivation | Sub-Contractor |

## Lean and Mean

End Object: Minimum possible costs  
Minimum possible complexity

How do you do that?  
Normalize EVERYTHING!  
Remove ALL redundancy - NO recurring concepts

Redundancy:

1. Unnecessary costs of duplication - waste.
2. Compensatory costs of discontinuity - Entropy  
(Entropy = energy not available for productive work)
  - a. Internal costs - operating expenses
  - b. External costs - damage control, litigation

Second law of thermodynamics - the aging process.  
Over time, the energy required to support the system (entropy) increases. At the point in time the energy required to support the system exceeds the energy in the system, the system dies. How do you remove entropy? Re-engineer the system to remove disorder. Take out all discontinuous duplication. Engineer for simplicity.

© 2000-2006 John A. Zachman, Zachman International

## Finding Redundancies

How do you discover recurring concepts?  
How do you "normalize" anything? CLASSIFY.

But - the classification scheme has to be "clean." You can't have mixtures (apples and oranges) in any category because you won't be able to detect redundancies. The schema has to be "normalized" - one fact in one place.

And - the schema has to be comprehensive. You must have a category for every concept or you won't find the duplication of concepts that are not classified.

That is, the schema has to be comprised of single variable, "primitive" categories. No mixtures (composites.) The schema has to be complete, the total possible set of categories.

For example, the Periodic Table.

Anything less than the total set would either, by definition, be DE-normalized (contain composite categories) or could not accommodate the totality of the concepts.

© 2000-2006 John A. Zachman, Zachman International

# The Framework

Primitive Models are architecture

Primitive models defined relative to the Enterprise are ENTERPRISE Architecture. Long term investments.

Composite Models are implementations

Composite models defined relative to one project are implementations. It is doubtful that you could define a composite, Enterprise-wide Model. It would be so complex, who could possibly understand it?. Composite models are short term implementations.

**YOU DON'T HAVE TO NORMALIZE ALL 30  
PRIMITIVE MODELS TO REALIZE SHORT TERM  
OPTIMIZATION BENEFITS!**

(Note: discontinuity in Columns 1, 3 and 6 will directly, negatively impact management's performance.)

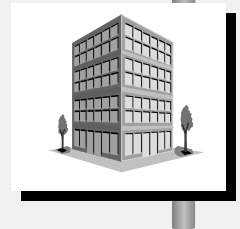
## POINT NO. 2

If you retain and maintain the primitive models, they are the baseline for managing change.

© 2000-2006 John A. Zachman, Zachman International

**Enterprise Architecture**

**Value Proposition**



© 1990-2006 John A. Zachman, Zachman International



## Industrial Age (Old)

"Better, Faster, Cheaper"

Computers do it the same way every time

(People make mistakes)

Computers run at machine speeds

(People run at people speeds)

Computers are cheaper

(Labor is more expensive than machines)

"Better, Faster, Cheaper"

"Justify" the acquisition of computers based on

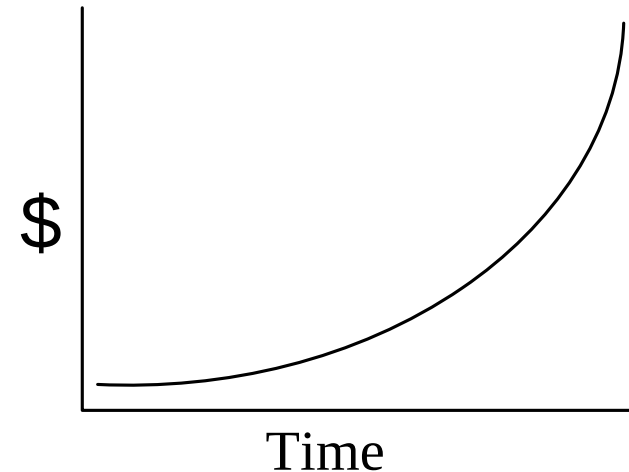
Cost-displacement -

how many people will it (they) replace?

Therefore, Value proposition:

"Cost-Justification"

## Short Term Investment



Expense-based, short term oriented, implementation-dominant, "cost-justified," "you start writing the code ... I'll go find out what the users have in mind."

Start manufacturing before you do any engineering.

**NO ARCHITECTURE**

Quick implementations. Consumables.

Point-in-time solutions. One time use.

"Pay me now or pay me later" - Scrap and rework.

## Information Age (New)

### Value Proposition for ARCHITECTURE

(Note: You can't "cost-justify" Architecture because it doesn't displace any costs.)

Architecture is an INVESTMENT that enables you to do things you otherwise would be unable to do ... specifically:

Alignment

Integration

Change (Flexibility)

Reduced time-to-market

Security Assurance

Value proposition: Asset Inventory

## Value Proposition for Architecture

### A. Alignment

The implemented enterprise reflects the intent of "the Owners."

(In manufacturing - this equates to "Quality" - "TQM")

### B. Integration

The data means the same thing to everyone.

Messages are successfully (and consistently) transmitted from node to node.

Everyone understands the objectives/strategy.

(The enablers of "empowerment.")

(This is standard, interchangeable parts.)

### C. Change Management (Flexibility)

Independent variables - baseline for managing change.

Retained, accumulated, Enterprise knowledge .

(Change with minimum time, disruption and cost.)

(This is "effectivity," change management.)

### D. I/S Responsiveness

Architecture plus "assemble-to-order" processes.

(This is reduced "time to market" - "Mass Customization.")

## Value Proposition for Architecture

### E. Security

Until now, "amateurs" ... (2004 & beyond) professional types of attackers, targeting crucial online systems.

Robert Clyde CTO, Symantec, 2003

... today's threat ... well-organized criminal syndicates employing sophisticated and structured attack techniques.

World Bank 2003

**Roger Schell, AESec Corp. roger.schell@aesec.com**

Dependent on computer tech. to prevent grave damage

Software of uncertain pedigree

Commercial software & open source easily subverted

Much overseas - software supplied by our competitors

Sometimes called the MLS (MultiLevel Security) problem

Prevents interconnection of networks

Impedes real-time sharing of information

**Subversion** demonstrated as the attacker's tool of choice

**Architecture for High Assurance MLS -**

**Major Issue is Verifiable Protection**

Presentation at StratCom 4-6-2004

***I submit - there is NEVER going to be verifiable protection for high assurance without EA! Some day ... !!!***

© 2000-2006 John A. Zachman, Zachman International

## Information Value

In addition to:

Alignment

Integration

Change

Reduced Time-to-Market,  
Security Assurance

the set of models that constitutes

Enterprise Architecture

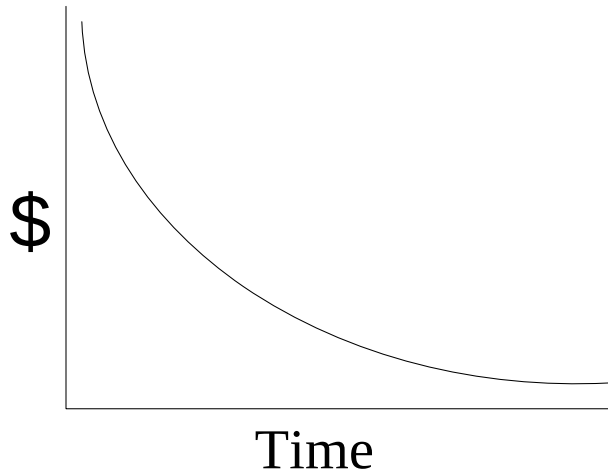
is the structured, explicit "Knowledge-Base" of the Enterprise against which the implicit, "tacit" knowledge can be mapped, classified, evaluated and/or transformed to "explicit" ...

... to give management the Information Age

Knowledge Advantage.

© 2000-2006 John A. Zachman, Zachman International

## Long Term Investment



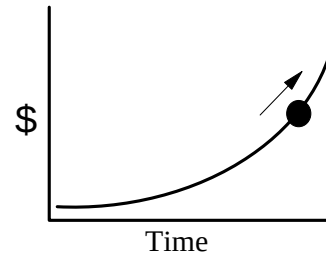
Long term, asset-based, integration (normalization) dominant, investment-oriented, engineered for reusability.

Do the Engineering BEFORE you start manufacturing.  
ARCHITECTURE

Create re-usable assets. Minimize scrap and rework.

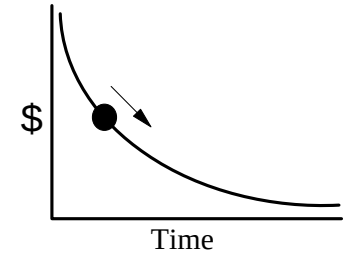
Takes up-front time and money for initial investment.

## Engineering Design Objectives



This is the short term value:

Implementation



These are long term values:

Alignment  
Integration  
Flexibility  
Interoperability  
Reduced Time-to-Market  
Quality  
Seamlessness  
Adaptability  
User-Friendliness  
Usability  
Reusability  
etc., etc.

## Compromise

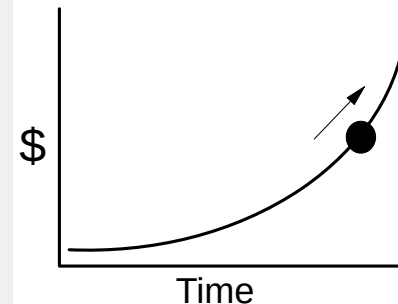
If you are going to do things that compromise the long term, best interests of the Enterprise, you probably should:

- A. Know that you are doing it,
- B. Know why you are doing it,
- C. Make every effort to mitigate the downstream effects of the compromise, and
- D. Make sure that everybody who would have reason to be unhappy later understands all of the above ... and actually participates in the compromise decision process.

The ENTERPRISE should be making the long term - short term trade-off decision ... **not** I/S.

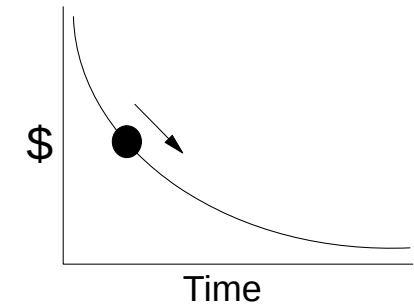
## Investment Curves

### Cost Justification (Expense-based)



**Short Term**

### Architecture Investment (Asset-based)



**Long Term**

Manufacture before it is  
engineered

Engineer before it is  
manufactured

Implementation

Integration

Point in time solutions

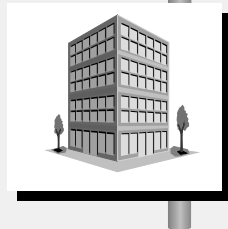
Assemble-to-order

Pay me now or  
pay me later

It takes too long and  
costs too much

**Enterprise Architecture**

**Cheaper and Faster**



## **Cheaper and Faster**

Using a top-down, Enterprise Architecture approach, a rigorous, enhanced Information Engineering Method, Three-Schema Data Architecture and CASE technology:

Cost per new entity type/RDBMS table  
reduced from >\$150,000 to <\$10,000.

Enterprise data handling labor reduced 50%.

Reduced development time of 25% through  
improved communication and conflict resolution.

Development time and cost reductions for every  
succeeding implementation of >50%,  
compounded, through reuse of database and  
application components with no modifications.

Reduced disk space for data (including history)  
of 20% - 80% through elimination of redundancy.

*Reference: Doug Erickson 614-751-5078*

© 2004-2006 John A. Zachman, Zachman International

## Cheaper and Faster

State of Ohio: Workers Compensation Board

Rates System                      1,030 entity types  
(operational - 2 1/2 years elapsed time)

Benefits Payments                720 e/t's (Reused 470)  
(operational)

Retro Rated Billing                230 e/t's (Reused 220)  
(operational)

4 years total elapsed time, no database failures,  
< 40 hours required maintenance

Health Provider Mgmt            415 e/t's (Reused 255)  
(under development)

Total Cost per entity type        \$25,000 (conservative)

includes legacy data cleansing

all data conversion costs

all interfaces with remaining legacy

no redundancy - reduced entropy

complete Enterprise alignment and integration

Total savings: 945 (reused) x \$25,000 = \$23,625,000

## Cheaper and Faster

Recent Package implementation: \$50,000/ET (conserv.)  
(no data cleansing, no data conversions, no legacy  
interfaces, added redundancy and 60% functionality)

Recent Custom Apps: \$100,000- \$150,000/Ent. Type  
typical legacy, redundant environment (re: entropy)

### Comparative Costs

Trad. Appln Devel 2395 e/t's x \$140,000 = \$335,300,000

Package                      2395 e/t's x \$50,000 = \$119,750,000

Ent. Arch.            2395 - 945 e/t's x \$25,000 = \$36,000,000

(and Enterprise Architecture approach

"aligned," low maintenance, no entropy)

Re: "Reusable code"

In three operational Systems:

6,128 action blocks            Avg. Reuse factor = 7.0

(Trad. A/D: Code, test, maintain 42,896 subroutines)

(attributable to granularity and precision of the data  
model, i.e. many processes use the same data.)

*Reference: Doug Erickson 1-614-751-5078*

## Cheaper and Faster

| State of Ohio |                              | Different State                                                                                          |
|---------------|------------------------------|----------------------------------------------------------------------------------------------------------|
| Workers Comp. | <i>Application</i>           | Child Welfare                                                                                            |
| IEF/CoolGen   | <i>Same CASE Tool</i>        | IEF/CoolGen                                                                                              |
| Architected   | <i>Different Methodology</i> | Classic                                                                                                  |
| 1030          | <i>Entity Types</i>          | 300                                                                                                      |
| 2.5 Years     | <i>Elapsed Time</i>          | 12 Years                                                                                                 |
| -             | <i>Development Costs</i>     | \$42 Mil.                                                                                                |
| \$25,000      | <i>Cost per Entity Type</i>  | \$140,000<br>(2 prime contractors<br>and one local cntrtr.<br>Estimating 3 more years<br>to enhance/fix) |

Reference: Doug Erickson 614-751-5078

© 2004-2006 John A. Zachman, Zachman International

## The REAL Benefit

From: Jim Tompkins (**Large Customer**)

To: Lauree Raica (**Chief Risk Officer**)

THANK YOU! THANK YOU!! THANK YOU!!!

This is great news for Frank Gates and The Service Assn. of Ohio. I appreciate so much your continuing efforts to help facilitate the improved turnaround time on these quarterly claim cost updates. I also want to express my appreciation to the "systems staff" at BWC in moving us forward with receiving the claims status and effective date information in the updated PIRS system. This will be a significant benefit.

From: Jim Romig (**Chief, Employer Relations**)

To: Admiral Jim Conrad (**CEO**)

Adm. Conrad, the IT Department did a great job in speeding up the turnaround time for quarter ending reserves being posted on Dolphin. What used to take 24 days now takes less than 10 . We have received several thank you's from TPAs since they are now able to get Sept. 30th data by Oct. 10th instead of Oct. 24th.

© 2004-2006 John A. Zachman, Zachman International



# The REAL Benefit

From: Leo Genders (**Dpty. Mgr. Appln. Dvlpmnt.**)

To: Rates & Payments Project team

This is excellent news and worthy of high praise. Great job to all involved. It is not often that an outside customer praises the internal IT Team!

From: Kevin Ribble (**Appln. Dvlpmnt. Mgr.**)

To: Nary Loganathan, Russel Marwitz (**Erickson Contractors**)

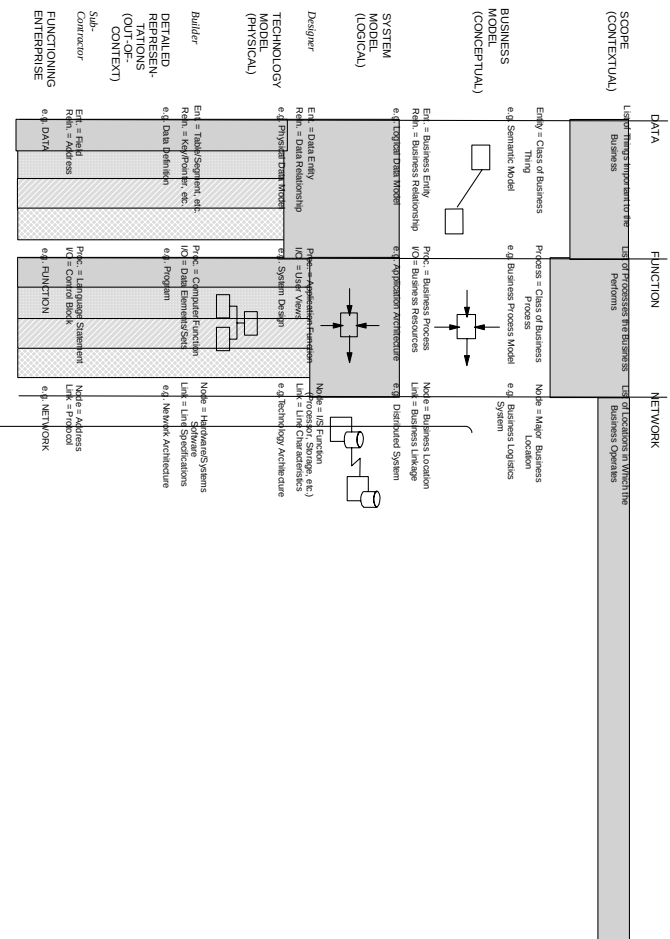
Great job on this! Not only have you improved service for our customers, but the significant decrease in processing time makes things much simpler for all of us in IT.

From: Nary Loganathan (**Erickson Contractor**)

To: Doug Erickson

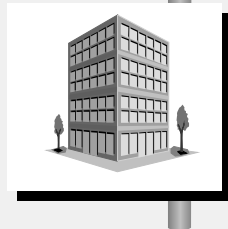
And the icing on the cake, tabular reserves completed in 11 hours yesterday night (ran for ~ 4.5 days in previous quarters) ... minor read statement change.

## Plan A (roughly)



## Architecture Versus Legacy

## A New Destination



## Industrial Age Products

It was not just about identifying the concepts (Row 1)  
It was not just about defining the requirements (Row 2)  
It was not just about designing the finished good (Row 3)  
It was not just about specifying the manufacturing (Row 4)  
It was not just about configuring the tooling (Row 5)  
It " " " assembling the final composite product (Row 6)

*It's about how you integrate and  
manage all the transformations.*

It was not just about engineering a Bill of Materials (Col. 1)  
It was not just about engineering the Functionality (Col. 2)  
It was not just about engineering the Geometry (Col. 3)  
It was not just about engineering the Ergonomics (Col. 4)  
It was not just about engineering the Timing (Col. 5)  
It was not just about engineering the Design Objectives (Col 6)

*It's about how you balance and  
integrate all of these together.*

## The Information Age

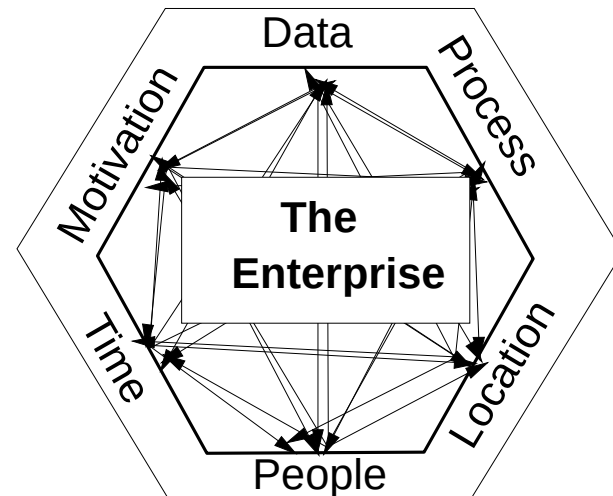
Actually, in the Information Age, it's not just about delivering one finished good, one implementation.

*It's about how rapidly you can fulfill different, custom, complete, holistic, new customer requirements, that is, produce different, unique Enterprise-wide "integrated" implementations with virtually zero defects (six sigma) so they run flawlessly 24 X 7 until they are replaced dynamically with the next new, flawless, Enterprise-wide, integrated implementation.*

This presumes **assembling** the Enterprise to order from a set of architectural primitives that is, "mass customization" of the Enterprise.

## The End State Vision

The End State Vision: You want to define each primitive model (Cell) from the perspective of the Enterprise, Enterprise-wide, at excruciating level of detail, defining the relationships horizontally across the Rows (as depicted below) and vertically down the Columns, from which to create the composite implementations by hard binding together only on demand, that is, assemble the Enterprise to order ... mass customize the Enterprise.



## Enterprise Architecture Plan

You are not going to get a set of Enterprise-wide, normalized, primitive models in inventory by accident ...

and they are not going to just happen or evolve out of the legacy.

There simply is "NO SILVER BULLET"

Fred Brooks 1980 ???????

It is going to take some time, money and deliberate action ...

## The Role of Science

"To take apart and to put together are different things. To confuse the two is grossly unscientific. For the beginning of Science is the realization that classification (*the primitives*), while absolutely necessary does not tell us any important fact about the nature of the thing being classified (*the composites*). Peter Drucker 1954  
(JAZ additions in italics)

"It is the function of science to discover the existence of a general reign of order in nature and to find the causes governing this order."  
Dmitri Mendeleev circa 1890

Mathematics: "God Created the Integers" Stephen Hawking 2005

Chemistry: The Periodic Table Mendeleev 1890

Biology: Taxonomy Somebody

Linguistics: Alphabet Anonymous

Navigation: Latitude and Longitude Somebody

Economics: Chart of Accounts Somebody

Physics: Laws of Motion Newton

Astronomy: Laws of Planetary Motion Kepler

etc., etc., etc.

Until someone discovers the underlying primitive constructs, putting things together is neither predictable nor repeatable.

Without classification (primitives) there is no science.

Everything is an instance (composite), trial and error.

## Conclusion

Primitive models are Architecture.  
Composite models are implementations.

The question is:  
where did the composite models come from?

If no primitive models exist and you are producing implementations, you are not doing Architecture. You are building Legacy. That was okay during the Industrial Age. My opinion is, that is not going to be okay in the Information Age. The Information Age imperative is going to be Architecture, primitive models for Enterprise Engineering and Manufacturing, assembling the Enterprise to order

... **mass-customization!**

## Enterprise Architecture Futures

"Someday ... the ENTERPRISE is going to HAVE TO HAVE all of these models made explicit, Enterprise-wide, horizontally and vertically integrated at excruciating level of detail" ...

... forget about Model Driven Architecture and all the systems falderol ... the ENTERPRISE is not going to be able to operate on a day-to-day basis without the models ...

... and I don't think it will have until the sweet bye and bye to get a lot of these in place!!

## Enterprise Architecture Futures

Therefore ...

We, the EA Profession, are going to have to learn a lot about how to build Columns 4, 5, 6 (as well as Column 3) primitive models, and

We, the EA Profession, are going to have to learn a lot about how to build Rows 1 and 2 primitive models, and

We, the EA Profession, are going to have to learn a lot about how to manage all of the models, keep them in sync and dynamically current, and

We, the EA Profession, are going to have to learn a lot about how to "assemble (the Enterprise) to order."

... and I don't think it will have until the sweet bye and bye to learn a lot of these things!!

## Zachman Framework Contributions

We, Zachman Framework Associates, will publish standard metamodels for the Profession Framework, the Product Framework and the Classification (Zachman) Framework (in addition to the 2005 published standards for the Enterprise Framework). (2007)

We, ZFA, will publish sample primitive models for each of the 36 Enterprise Framework Cells. (2007)

We, ZFA, will release a Generalized Enterprise Modelling language workbench from which Rows 4 and 5 primitive architecture models as well as composite implementation models can be generated (functional "Model Driven Architecture"). (2007)

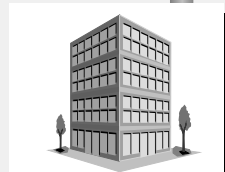
We, ZFA, will offer Certification Services to certify courses, models, methodologies and tools as Zachman Framework compliant. (2007)

We, ZFA, will offer Model Driven Enterprise Architecture implementation services for small businesses based on the Generalized Enterprise Modeling workbench. (2008)

We, ZFA, will *consider* producing a full-function Repository based on the Zachman Framework Standards. (2008)

## Enterprise Architecture

## Conclusions



## Zachman Propositions

1. The reasons you do Enterprise Architecture have to do with **complexity and change**. You cannot create a complex object (including an Enterprise) if you can't describe it ... and once you get it built and want to change it, the descriptive representations (architecture) constitute the base-line for changing it (whatever it is).
2. The Framework for Enterprise Architecture (the "Zachman Framework") is a classification scheme for descriptive representations ... descriptive representations of **anything**. (I learned about it from looking at descriptive representations of airplanes, buildings, locomotives, battleships, etc.) I applied the same logical schema to **Enterprises**. The Framework has **nothing to do** with information systems unless the Enterprise has some stored programming devices and electronic media installed, in which case, those technologies will be described in **Row 4** (not Row 1, nor Row 2 nor Row 3). The Rows 1, 2 and 3 models are descriptive of the **Enterprise** independent of **any** implementation technologies.
3. The "Zachman Framework" is a "normalized" schema - one (meta) fact in one place. It implies **nothing** about implementation process (that is, methodology: top-down, bottom-up, left-to-right, right-to-left or where to start.)

## Propositions (cont)

4. It can be used to help you think about (analyze) anything ... the broader you define the boundary of the analytical target, the more leverage you get on integration, reusability, interoperability, etc., of the end object (e.g. "Enterprise") but the more complex the analysis ... and conversely, the narrower you define the boundary, the simpler the analysis, but the less leverage you are going to get on integration, reusability, interoperability, etc.
5. It is okay to attempt to after the fact, post-integrate parts that you manufactured but that don't fit together ... but you can only "integrate" (interface) cosmetic anomalies ... it is like putting lipstick on a pig. It makes the pig look better but it doesn't change the fact that it is a pig.
6. You don't have to build out all the models defined by the Framework, Enterprise-wide at excruciating levels of detail, before you can implement something ... you just have to understand that whatever slivers (vertical or horizontal) of whatever cells you are not building out (making explicit), you are making assumptions about, that is, you are assuming risk ... risk of defects ... ultimately, risk of "scrap and re-work."

## Propositions (cont)

7. You don't have to build Enterprise-wide models for implementations but you'd better pay attention to Enterprise-wide discontinuities in Column 1 (meaning), Column 3 (connectivity) and Column 6 (motivation) because if, after you get a bunch of things implemented (like, "the legacy"), and the data doesn't mean the same thing to everyone (is not "integrated"), the network is instable and inflexible and management is not able to administer the objectives/strategies (business rules) consistently, they (management) are going to be frustrated. (Are they frustrated?)
8. If you are not observing the engineering design principles as related to the primitive (cell) models, you are **not** going to realize the engineering design objectives (alignment, integration, interoperability, reusability, flexibility, reduced time-to-market, etc., etc., etc.) Composite (multi-cell) models are required for implementations. Primitive (single-cell) models are required to engineer alignment, integration, reusability, etc.



## Propositions (cont)

9. If you are not building (and storing, managing and changing) **primitive** models you are not doing "Architecture" ... you are doing implementations.

10. Until you have some (primitive) models stored somewhere in such a fashion that you can find them and reuse their components, you are by definition, a "job shop" (a "waterfall") manufacturing "custom" products. That is, you are never going to reduce time-to-market for anything substantive until you have something (parts, i.e. "**primitives**") that are designed to be reused in more than one implementation (composite) anywhere in the Enterprise, in inventory, stored in such a fashion that they can be located, **before** you get the order for the implementation.

Note:

If anyone refers to the "Zachman Framework" and says something inconsistent with these propositions, they either don't understand the logic of the Framework or they don't understand the implications of the logic.

© 2001-2006 John A. Zachman, Zachman International

## Jay W. Forrester

" Although social systems are more complex than physical systems, they belong to the same class of high-order, non-linear, feedback systems as do physical systems.

People do not accept the idea that families, corporations, and governments belong to the same class of dynamic structures as do chemical refineries and autopilots for aircraft.

"Organizations built by committee and intuition perform no better than would an airplane built by the same methods ... As in a bad airplane design, which no pilot can fly successfully, such badly designed corporations lie beyond the ability of real-life managers.

"I anticipate future management schools devoted to 'enterprise design'. ... A fundamental difference exists between an enterpriseoperator and an enterprise designer. A manager runs an organization, just as a pilot runs an airplane. Success of a pilot depends on an aircraft designer who created a successful airplane. ...who designed the corporation that a manager runs?"

*"Designing the Future" by Jay W. Forrester 12/15/98*

© 2006 John A. Zachman, Zachman International

## 1965 Systems Problems

1. Didn't meet Requirements. (not "aligned")
2. The data was no good:
  - Not consistent from system to system.
  - Not accurate.
  - Not accessible.
  - Too late.
3. Couldn't change the system. (Inflexible)
4. Couldn't change the technology. (Not adaptable)
5. Couldn't change the business. (Couldn't change the system or the technology so couldn't change business.)
6. Little new development (80% \$ for maintenance)
7. Took too long.
8. Cost too much.
9. Always over budget.
10. Always missed schedules.
11. DP budget out of control.
12. Too complicated - can't understand it, can't manage it.
13. Just frustrating.

*(Adapted from Doug Erickson)*

## 2006 Systems Problems

1. Don't meet Requirements. (not "aligned")
2. The data is no good:
  - Not consistent from system to system.
  - Not accurate.
  - Not accessible.
  - Too late.
3. Can't change the system. (Inflexible)
4. Can't change the technology. (Not adaptable)
5. Can't change the business. (Can't change the system or the technology so can't change business.)
6. Little new development (80% \$ for maintenance)
7. Takes too long.
8. Costs too much.
9. Always over budget.
10. Always missed schedules.
11. IT budget out of control.
12. Too complicated - can't understand it, can't manage it.
13. Just frustrating.

*(Adapted from Doug Erickson)*

## It's Funny ...

COBOL didn't fix those problems!

MVS didn't fix those problems!

Virtual Memory didn't fix those problems!

IMS, DB2, Oracle, Sybase, Access, Fortran, PL/1, ADA, C++, Visual Basic, JAVA 2, 360's, 390's, MPP's, DEC VAX's, H200's, Crays, PC's, MAC's, Distributed Processing, didn't fix those problems!

Word, Excel, Powerpoint, Outlook Express, eMAIL, DOS, Windows 95, 98, 2000, NT, ME, XP, Unix, Linux, Object Oriented, COM, DCOM, CORBA, EDI, HTML, XML, UML, the Internet, B2B, B2C, Portals, Browsers didn't fix those problems!

IEF, IEW, ADW, ERWIN, POPKIN, Rational, PTECH, Rochade, Platinum, Design Bank, Data Warehouse, SAP, Baan, Peoplesoft, Oracle Financials, BSP, ISP, EAP, EAI didn't fix those problems!

And, I doubt that Web Services, .Net, Websphere, Extreme Programming, Service Oriented Architecture or Component Development (whatever that is) is going to fix the problems.

IT MAKES ONE WONDER IF THERE ACTUALLY IS A TECHNICAL SOLUTION TO THE PROBLEM!!!

## Engineering Problem

I'm not saying that there is anything wrong with any of these technologies.

In fact, any or all of them may well be very good ...

In fact, you may not be able to solve the Enterprise problem without employing some of these technologies.

However,

The Enterprise problem is an ENGINEERING problem, NOT a technical problem.

My perception is that it is going to take actual work, ENGINEERING work, to solve the problem. My plan would be to start building out models, PRIMITIVE models, engineering them for alignment, integration, flexibility, reduced time-to-market, etc., etc., etc.

What would be YOUR plan for solving the problems???

# Framework Resources

## Zachman Enterprise Architecture:

Framework Fundamentals by John A. Zachman - 2 Days

Framework Implementation by Stan Locke - 2 Days

Enterprise Architecture Planning - 3 Days

Fundamentals by John A. Zachman 1 1/2 Days

Planning Methodology by Sam Holcman 1 1/2 Days

## Seminars Planned for 2007

Zachman Enterprise Architecture Master Class

4 Days by John A. Zachman and Stan Locke

Zachman Enterprise Architecture: Practicalities and Implementations

4 Days by John A. Zachman and Stan Locke

Enterprise Architecture Planning

3 Days by John A. Zachman and Sam Holcman

Resources at [www.ZachmanInternational.com](http://www.ZachmanInternational.com)

1. "The Zachman Framework: A Primer for Enterprise Engineering and Manufacturing" book by J. A. Zachman
2. 25 Articles by John A. Zachman
3. Zachman Framework Standard Metamodel - Personal License (no charge)
4. 10 Presentations from the Proceedings of the 2006 DAMA Conference Zachman Track (no charge)